

TESTING DE APLICACIONES

Martín Benítez Potochek

Materia: Testing de Aplicaciones

Docente: Laime Huanca Abel Israel

Fecha: 03/05/2026

Título: TPO2 – Automatización de pruebas y pipeline CI/CD

Índice

Parte 1 – Diseño del escenario de prueba.....	3
Parte 2 – Automatización de pruebas.....	3
Parte 3 – Implementación del pipeline (CI/CD).....	4
Parte 4 – Evidencia y análisis.....	5
Parte 5 – Reflexión.....	6
Conclusiones personales.....	6
Propuestas de futuras mejoras.....	6
Bibliografía.....	7

Parte 1 – Diseño del escenario de prueba

Para el TP elegí hacer una calculadora simple en Python con cuatro operaciones: sumar, restar, multiplicar y dividir. Es una funcionalidad fácil de probar y permite cubrir los tres tipos de casos que pide la consigna.

Escenario: el usuario realiza operaciones aritméticas básicas.

Casos de prueba:

- **Caso exitoso:** sumar(2, 3). Resultado esperado: 5.
- **Caso de error:** dividir(10, 0). Resultado esperado: lanza un error porque no se puede dividir por cero.
- **Caso borde:** sumar(-5, 3). Resultado esperado: -2 (suma con un número negativo).

Parte 2 – Automatización de pruebas

Los tres casos se automatizaron con pytest. Cada test es una función que llama a la calculadora y verifica el resultado con un assert.

```
import pytest
from app.calculadora import sumar, dividir

def test_sumar_dos_numeros():
    assert sumar(2, 3) == 5

def test_dividir_por_cero():
    with pytest.raises(ValueError):
        dividir(10, 0)

def test_sumar_con_negativos():
    assert sumar(-5, 3) == -2
```

Parte 3 – Implementación del pipeline (CI/CD)

El pipeline está configurado con GitHub Actions, en el archivo **.github/workflows/tests.yml**. Cada vez que se hace un push al repositorio, GitHub corre los tests automáticamente y muestra si pasaron o no.

Pasos que ejecuta el pipeline:

- Bajar el código del repositorio.
- Instalar Python.
- Instalar las dependencias (pytest).
- Ejecutar los tests con pytest.
- Subir el reporte HTML como artefacto descargable.

Parte 4 – Evidencia y análisis

Capturas del pipeline ejecutándose:

[Captura: captura1_actions.png – Pestaña Actions con el run en verde.]

[Captura: captura2_steps.png – Detalle del run con los pasos en verde.]

[Captura: captura3_logs.png – Logs del paso de pytest mostrando los tests OK.]

[Captura: captura4_artefacto.png – Artefacto del reporte disponible para descargar.]

Resultado: los 3 tests terminaron OK.

Repositorio: <https://github.com/martinbep/tpo2-testing-aplicaciones>

¿Qué ventaja aporta automatizar pruebas dentro del pipeline?

Que los tests corren solos cada vez que se sube código. Así nadie se olvida de probar antes de subir y los errores aparecen al toque.

¿Qué impacto tiene esto en la calidad del software?

Mejora porque evita que código roto llegue a la rama principal. Cada cambio queda validado antes de quedar fijo en el proyecto.

Parte 5 – Reflexión

1. ¿Qué beneficios tiene automatizar pruebas frente a hacerlas manualmente?

Son más rápidas y se ejecutan siempre igual. A mano uno puede saltarse un caso, automatizado no.

2. ¿Qué dificultades encontraste al implementar el pipeline?

Lo más complicado fue armar el archivo del pipeline, sobre todo respetar la indentación porque cualquier espacio mal puesto rompe el workflow.

3. ¿Dónde aplicarías este enfoque en un entorno real?

En cualquier proyecto donde más de una persona toque el mismo código. Así cada cambio queda validado antes de mergearlo y se evitan bugs en producción.

Conclusiones personales

El TP me sirvió para entender en la práctica cómo funciona la automatización de pruebas. Una cosa es leerlo en la teoría y otra es ver el tilde verde aparecer solo en GitHub después de hacer un push.

Propuestas de futuras mejoras

Sumaría más tests para cubrir más casos, agregaría medición de cobertura del código y probaría la calculadora en distintas versiones de Python.

Bibliografía

TOLEDO, Federico. Capítulo: automatización de pruebas funcionales. Introducción a las pruebas en los sistemas de información. Montevideo, Uruguay: Abstracta, 2014.

Documentación oficial de pytest: <https://docs.pytest.org>

Documentación oficial de GitHub Actions: <https://docs.github.com/actions>